



DSA – Sec A

Lab – 12

<u>Instructor</u>	<u>Mr. Shehryar</u>
<u>Lab Instructors</u>	• Rimsha Aamir • Adeel Zafar • Muhammad Taha

Objective: This lab aims to deepen students' understanding of tree structures and efficient sorting mechanisms. Students will learn to implement and manipulate AVL Trees and Heaps and understand the concept of disk-based sorting algorithms.

Lab Tasks

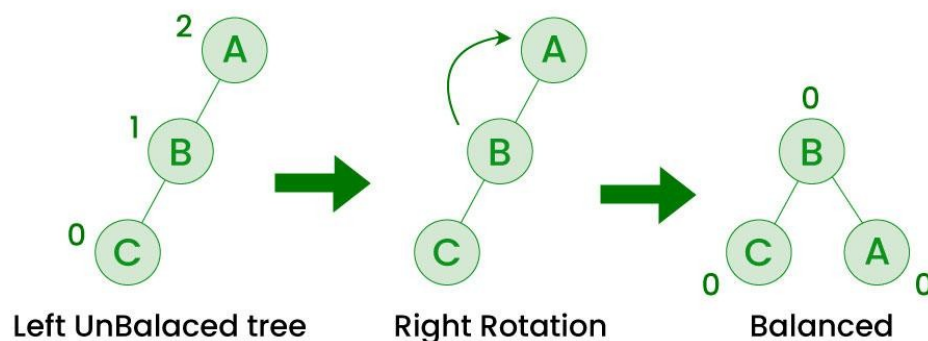
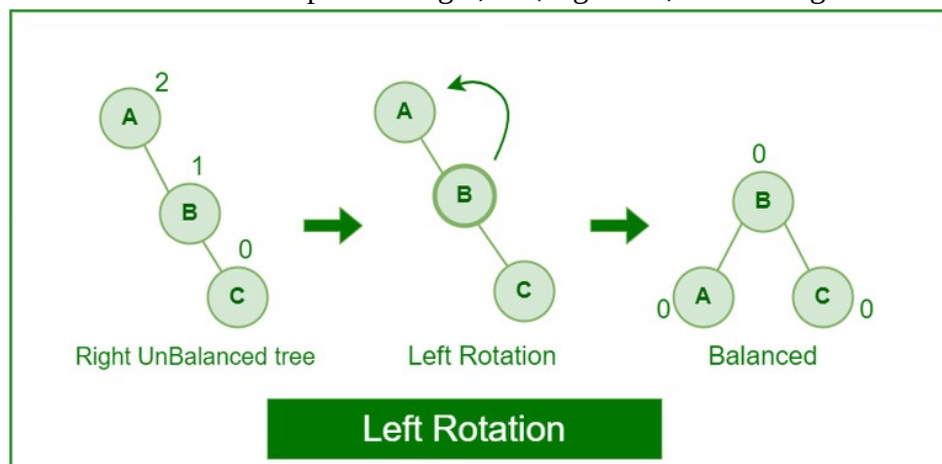
Task 1: AVL Trees

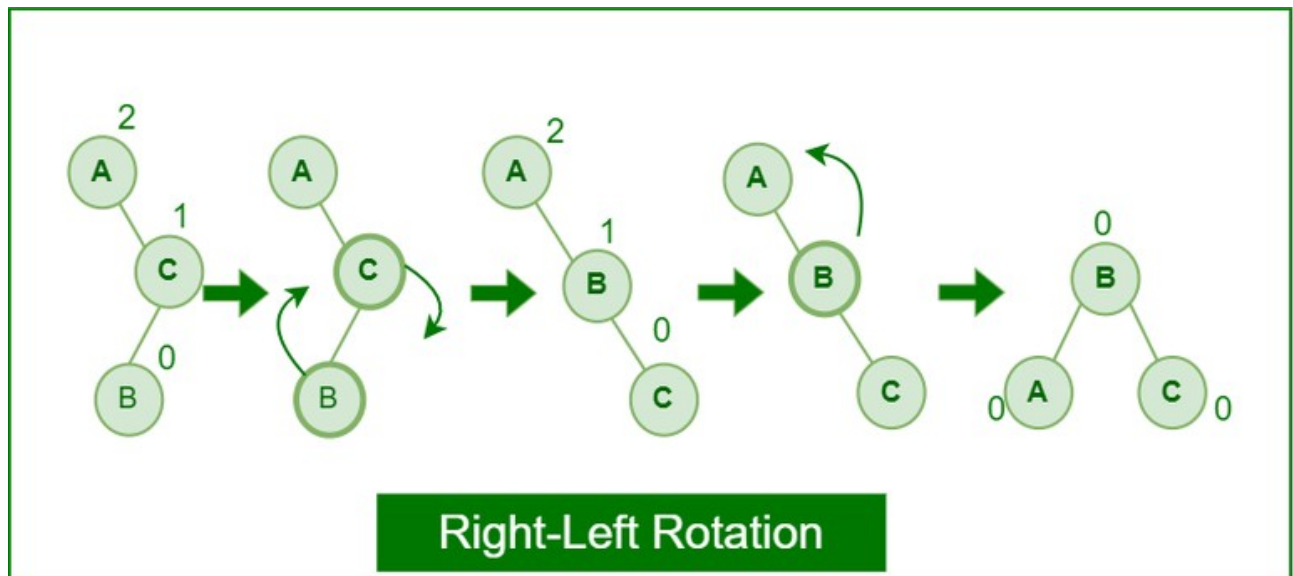
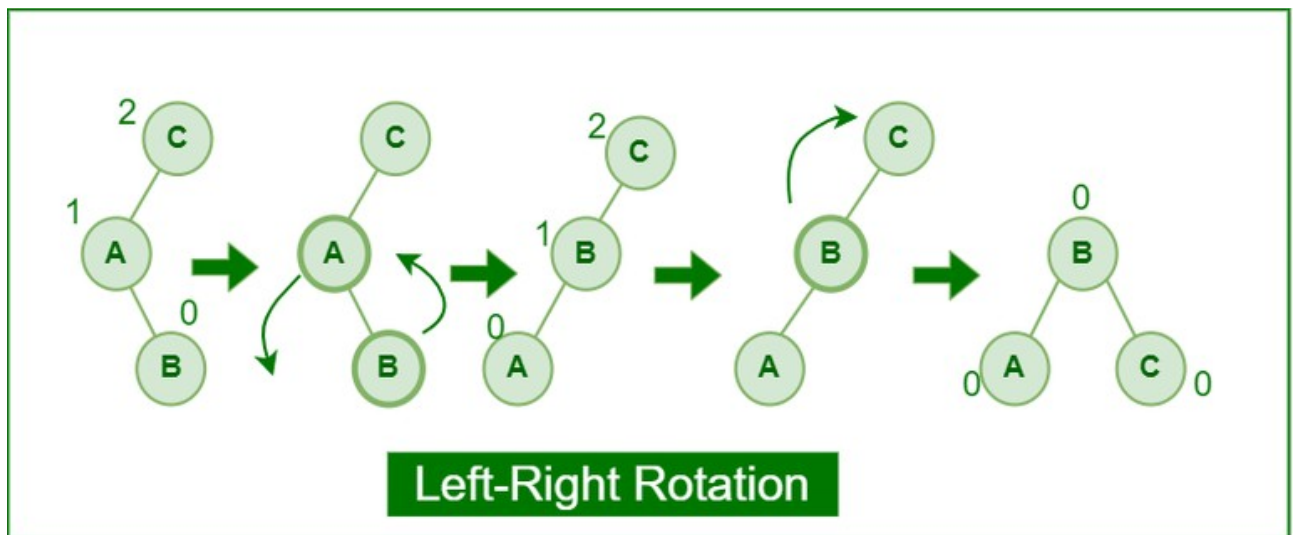
Objective: To understand and implement self-balancing binary search trees (AVL Trees), and analyze their importance in maintaining balanced trees for efficient operations.

Instructions:

1. Task Requirements:

- Implement a class `AVLTree` that includes:
 - A method to **insert** elements while ensuring the tree remains balanced.
 - A method to **delete** elements with re-balancing.
 - **Rotations:** Implement right, left, right-left, and left-right rotations.





- A method to calculate and update the **height** of nodes after each insertion or deletion.
 - Implement (any 2) **pre-order, in-order, and post-order traversal** methods for the AVL Tree.
 - Conduct test cases to validate the balance of the tree after each insertion and deletion.
- 2. Output:**
- Print the tree after a series of insertions and deletions.
 - Display the height of the AVL Tree after each operation.
 - Demonstrate traversal outputs.
-

Task 2: Heaps

Objective: To learn about Heaps, focusing on their structure, operations, and applications in priority queues.

Instructions:

1. Task Requirements:

- Implement two classes **MinHeap** and **MaxHeap** that support the following operations:
 - **Insertion:** Insert an element and maintain heap property.
 - **Deletion:** Remove the root element and maintain heap structure.
 - **Heapify:** Implement `heapify` function to adjust heap structure during insertions and deletions.
 - **Peek:** Implement a `peek` method to get the minimum or maximum element (depending on heap type) without removing it.
 - **Extract:** Implement a function to extract the root element.

2. Output:

- Show the structure of the heap after each insertion and deletion.
-

Task 3: Disk Sort

Objective: To understand disk-based sorting, suitable for large datasets that do not fit into memory.

Instructions:

1. Introduction:

- Explain the concept of **Disk Sort** (also known as **External Sort**) and why it is necessary for large data.
- Describe the **Merge Sort** approach, commonly used in disk-based sorting.

2. Task Requirements:

- Implement a simplified version of Disk Sort using the **External Merge Sort** algorithm.
- The implementation should:
 - Divide a large dataset into smaller chunks that can fit into memory.
 - Sort each chunk individually in memory.
 - Merge the sorted chunks to produce a fully sorted dataset.
- **Assume a dataset with 1 million integers** that cannot fit in memory all at once. Simulate disk-based sorting by dividing the data into chunks of 10,000 integers, sort each chunk, and then merge.

3. Output:

- Show intermediate steps of chunking and sorting each chunk.
- Display the first 50 and last 50 sorted elements from the final output.